Classes Abstratas

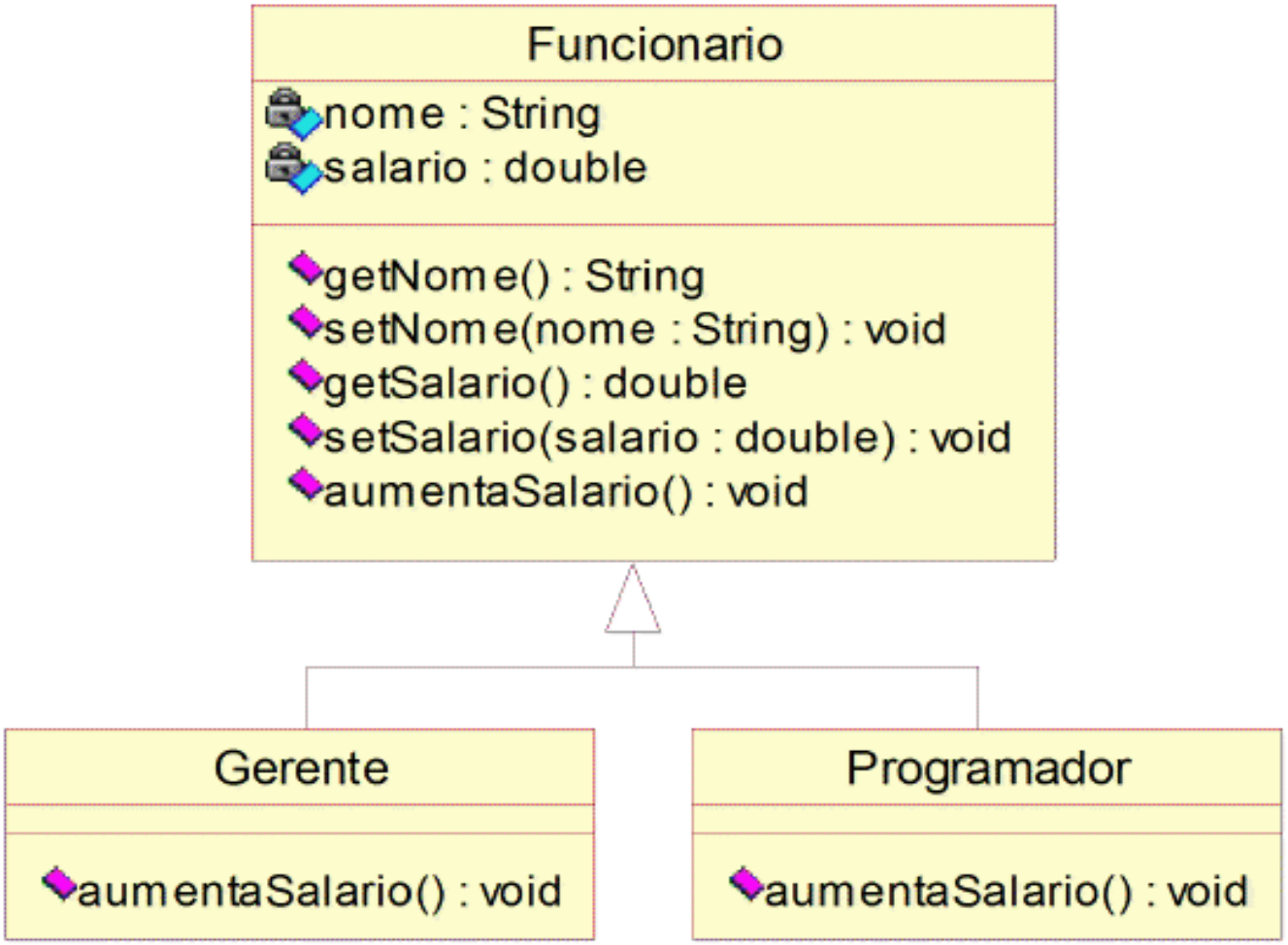
Orientação a Objetos – DCC025

2012.3

Classes Abstratas

- Fato
  - Em alguns casos, uma superclasse torna-se tão geral que acaba sendo vista como um modelo para outras classes e não como uma classe com instâncias específicas que são usadas.

Classes Abstratas



Classes Abstratas

- Definição
  - Uma classe abstrata é uma classe que não pode ser instanciada e possui nenhum, um ou vários métodos abstratos.
  - Um método abstrato é um método sem corpo cuja assinatura é precedida da palavra abstract e que não possui implementação (corpo vazio)

Classes Abstratas

- Exemplo:

```
public abstract class Animal
{
        abstract public void comer();
}
```

Declaração de classe abstrata
Com um método abstrato.

Classes Abstratas

- Definição
  - Somente são utilizadas como superclasses em hierarquias de herança.
  - Logo, são também chamadas de **superclasses** abstratas.
  - Métodos declarados como abstratos em uma superclasse devem, obrigatoriamente, ser implementados na(s) subclasses(s).
    - Erro de compilação, caso contrário!

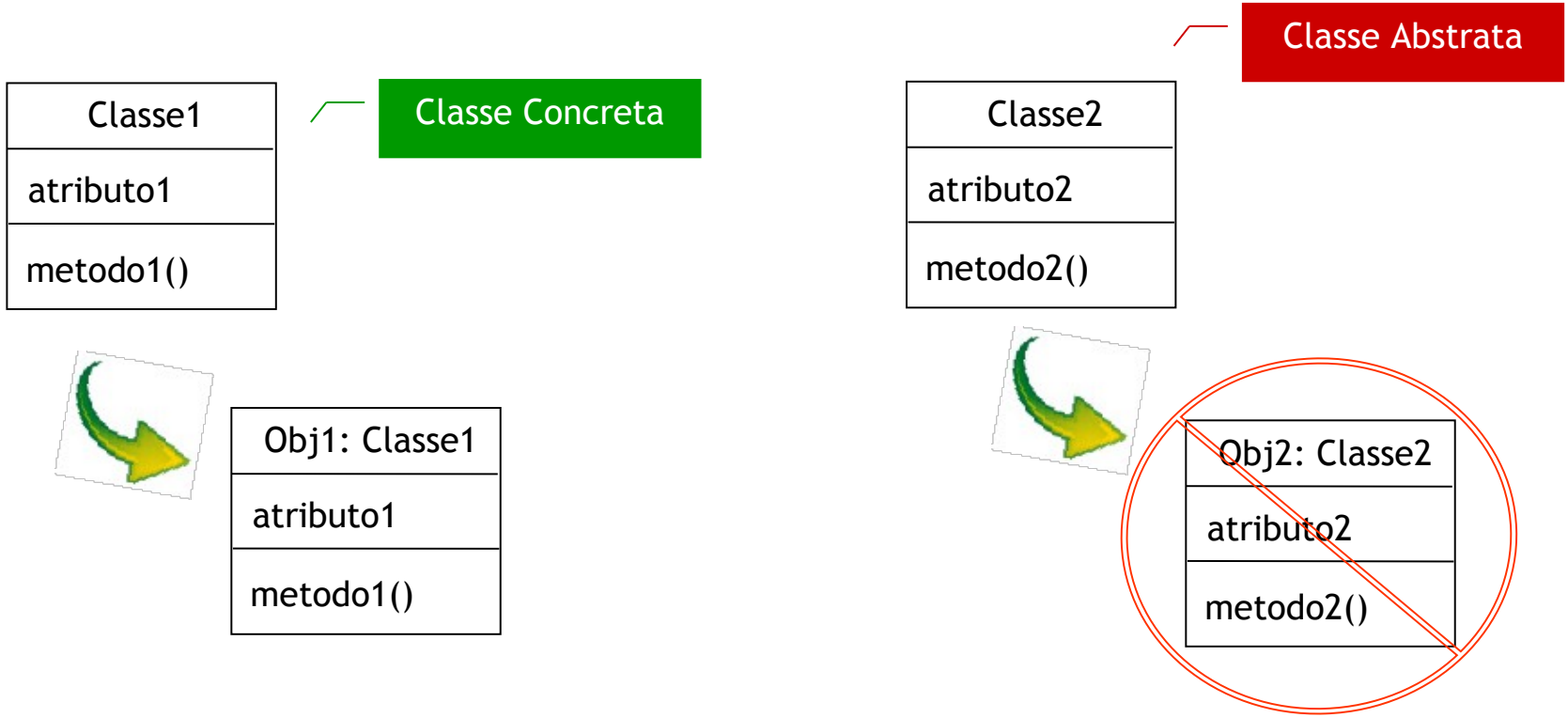
Classes Abstratas

- Definição
  - Classes abstratas são classes que não podem ter instâncias diretas.
    - Ex: Se X é uma classe abstrata, não se pode executar o código a seguir:

```
X objeto1 = new X();
```

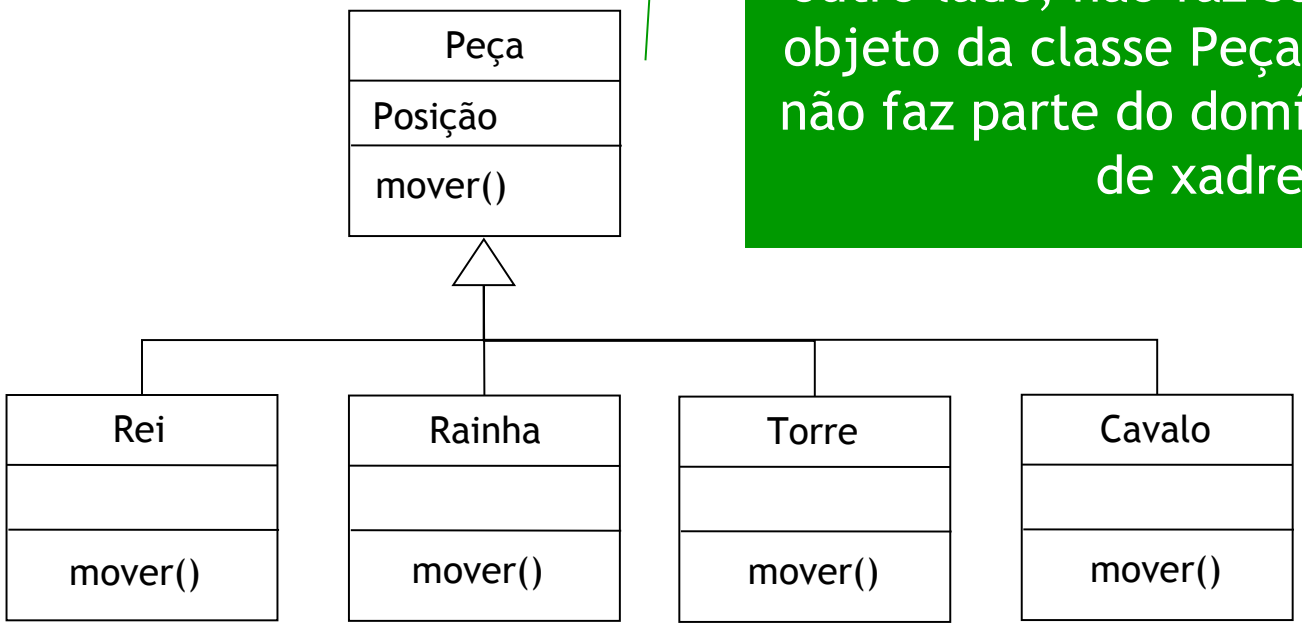
    - Contudo, suas subclasses podem ser instânciadas.
  - Por outro lado, uma classe concreta é uma classe instanciável – pode ter instâncias diretas.

Classes Abstratas



Classes Abstratas

Faz sentido criar objetos do tipo Rei, Rainha, etc., pois eles existem no domínio de um jogo de xadrez. Por outro lado, não faz sentido criar um objeto da classe Peça, pois a mesma não faz parte do domínio de um jogo de xadrez.



OBS: não há proibição em se criar um objeto da classe Peça. Contudo, como tal criação não faz sentido (ela só serve de molde para as subclasses), insere-se uma "proibição" declarando-a como sendo abstrata

Classes Abstratas

- Entendendo Classes Abstratas
  - Suponha que no banco todas as contas possuem um tipo específico, por exemplo, conta poupança, conta corrente ou conta salário e que elas são modeladas dentro do nosso sistema pelas seguintes classes:

Classes Abstratas

```
class Conta {
    private double saldo;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

```
class ContaCorrente extends Conta {
    private double limite;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

```
class ContaPoupanca extends Conta {
    private int diaDoAniversario;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

Classes Abstratas

```
class Conta {
    private double saldo;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

A classe Conta não define uma conta de fato - é apenas um molde

```
class ContaPoupanca extends Conta {
    private int diaDoAniversario;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

```
class ContaCorrente extends Conta {
    private double limite;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

Para cada conta do domínio do banco devemos criar um objeto da classe correspondente ao tipo da conta.

Classes Abstratas

```
class ContaPoupanca extends Conta {
    private int diaDoAniversario;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

Por exemplo, se existe uma conta poupança no domínio do banco devemos criar um objeto da classe ContaPoupança

Faz sentido criar uma classe ContaPoupança pois existem contas poupanças no domínio do banco.

Classes Abstratas

```
class Conta {
    private double saldo;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

Suponha que o banco ofereça extrato detalhado das contas e para cada tipo de conta as informações e o formato desse extrato detalhado são diferentes. Além disso, a qualquer momento o banco pode mudar os dados e o formato do extrato detalhado de um dos tipos de conta.

Não vale a pena ter uma implementação de método na classe Conta para gerar extratos detalhados. Porque?

Classes Abstratas

```
class Conta {
    private double saldo;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

Poderíamos, simplesmente, não definir nenhum método para gerar extratos detalhados na classe conta.

Quais seriam os problemas disso?

Classes Abstratas

- Métodos Abstratos
  - Para garantir que toda classe concreta que deriva direta ou indiretamente da classe CONTA tenha uma implementação de método para gerar extratos detalhados e além disso que uma mesma assinatura de método seja utilizada, devemos utilizar o conceito de métodos abstratos.

Métodos Abstratos

```
class Conta {
    private double saldo;

    // MAIS ATRIBUTOS E MÉTODOS
}
```

```
abstract class Conta {
    private double saldo;

    // GETTERS AND SETTERS

    public abstract void imprimeExtratoDetalhado();
}
```

Ao definir um método abstrato em uma claasse, devemos, obrigatoriamente, declar a classe como abstrata

Classes Abstratas

```
class ContaPoupanca extends Conta {
    private int diaDoAniversario;

    public void imprimeExtratoDetalhado() {
        System.out.println("EXTRATO DETALHADO DE CONTA POUPANÇA");

        SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy HH:mm:ss");
        Date agora = new Date();

        System.out.println("DATA: " + sdf.format(agora));
        System.out.println("SALDO: " + this.getSaldo());
        System.out.println("ANIVERSÁRIO: " + this.diaDoAniversario);
    }
}
```

```
// ESSA CLASSE NÃO COMPILA
class ContaPoupanca extends Conta {
    private int diaDoAniversario;

}
```

Toda classe concreta que deriva direta ou indiretamente da classe CONTA será obrigada a ter uma implementação desse método. Caso contrário a classe não compila.

Classes Concretas

```
1 public class Teste {
2
3     public static void main(String args[]){
4
        Classe1 Obj1 = new Classe1();
      }
```

Instanciação da Classe1

```
1 public class Classe1 {
2
3     private int atributo1;
4
5     public void metodo1(){
6     }
7 }
```

Classe Concreta Classe1

Classes Abstratas

```
1  public class Teste {
2
3      public static void main(String args[]){
4
5          Classe1 Obj1 = new Classe1();
6
7      }
8  }
```

Cannot instantiate the type Classe1

```
1  abstract public class Classe1 {
2
3      private int atributo1;
4
5      abstract public void metodo1();
6  }
```

Erro ao tentar instanciar objeto da classe 1

Classes abstratas não podem ser instanciadas

Classes Abstratas

```
1  abstract public class Classe1 {
2
3      private int atributo1;
4
5      abstract public void metodo1(){
6
7      }
8  }
9
10
```

Abstract methods do not specify a body

2 quick fixes available:

- Remove 'abstract' modifier
- Remove method body

Métodos abstratos não podem ter “corpo” - não importa se o mesmo estiver vazio

Métodos Abstratos

- Métodos Abstratos
  - São apenas declarados (com seu nome, modificadores, tipo de retorno e lista de argumentos), não tendo um corpo que contenha os comandos da linguagem que o método deve executar.

Métodos Abstratos

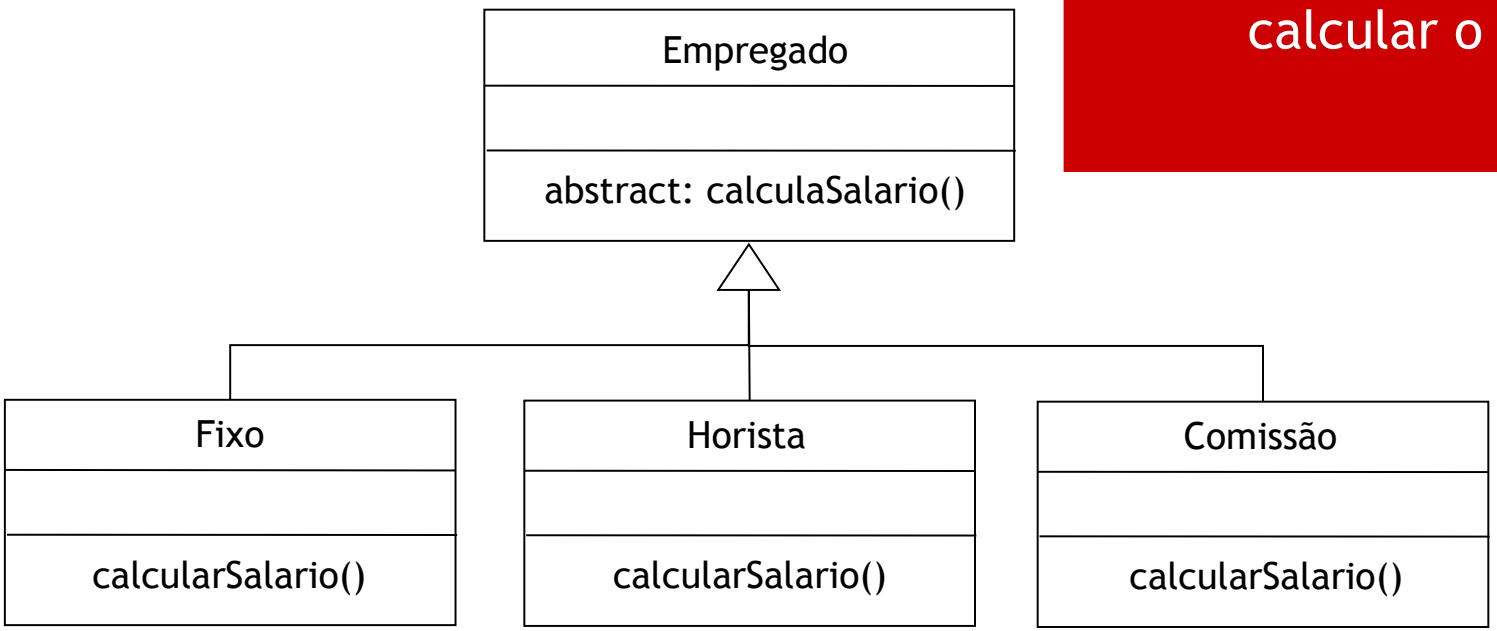
- OBS: se uma classe contém um método declarado como abstrato, as classes que herdarem desta deverão, obrigatoriamente, implementar o método abstrato com o nome, modificador, tipo de retorno e argumentos declarados na classe ancestral.

Métodos Abstratos

- Métodos Abstratos
  - OBS: se uma classe tiver algum método abstrato, a classe também deverá, obrigatoriamente, ser declarada com o modificador “abstract”
    - Contudo, uma classe pode ser declarada como abstrata mesmo que nenhum método tenha sido, explicitamente, declarado como abstrato. Ou seja, uma classe pode ter só métodos concretos (exemplos são os métodos get e set).

Classes Abstratas

Ao declarar o método como abstrato, o programador força que todas as subclasses implementem uma forma de calcular o salário



Classes Abstratas

```
1 abstract public class Funcionario {
2
3     abstract public int calcularSalario();
4
5 }
```

```
1 public class Fixo extends Funcionario{
2
3
4
5 }
6
```

The type Fixo must implement the inherited abstract method Funcionario.calcularSalario()

2 quick fixes available:

- Add unimplemented methods
- Make type 'Fixo' abstract

```
1 public class Fixo extends Funcionario{
2
3     public int calcularSalario(){
4         return 0;
5     }
```

Corpo da classe Fixo não contém o método calcularSalario - Logo, é exibido uma mensagem de erro. A classe Fixo é obrigada a implementar tal método (seu corpo não pode ser vazio)

Classes Abstratas

```
1 abstract public class Funcionario {
2
3     abstract public int calcularSalario();
4
5 }
```

```
1 public class Fixo extends Funcionario{
2
3     public int calcularSalario(){
4         return 0;
5     }
6 }
```

```
1 public class Horista extends Funcionario{
2
3     public int calcularSalario(){
4         return 0;
5     }
6 }
```

O método calcularSalario() é processado polimorficamente

Classes Abstratas

```
1 abstract public class Funcionario {
2
3     abstract public int calcularSalario();
4
5 }
```

Porque declarar calcularSalario como abstract?

Não faz sentido fornecer uma implementação desse método na classe Funcionario. Não podemos calcular os vencimentos para um Funcionário geral – é preciso conhecer o tipo de empregado para, então, determinar a cálculo apropriado dos vencimentos

Classes Abstratas

- Objetivo
  - O objetivo de criarmos classes abstratas é para encapsular outras classes com comportamento comum. Elas podem surgir naturalmente na modelagem ou serem criadas para promover o reuso.

Exercícios

Exercício 01

- Seu objetivo é, considerando os conceitos de OO, projetar um programa para o cálculo de imposto de renda de contribuintes do tipo Pessoa Física (PF), Pessoa Jurídica (PJ) e Pessoa Especial (PE).
- Toda PF possui CPF e renda bruta. Toda PJ possui CNPJ e renda bruta. Toda PE possui NS (Número Social) e renda bruta.
- O cálculo do imposto, realizado através do método calcImposto(), deve ser feito da seguinte forma: (i) PJ: o imposto corresponde a 10% da renda bruta da empresa, (ii) PF: conforme tabela apresentada em sala e (iii) PE: o imposto corresponde a 10% de renda bruta.
- Considere uma classe ReceitaFederal, que recebe um conjunto de contribuintes e, para cada um, imprime um relatório informando a identificação do mesmo, a renda bruta e a renda descontada dos imposto.
- Crie os construtores das classes.
- Crie uma classe **Principal** que teste seu programa (deve ser criado uma pessoa de cada tipo).

Exercício 02

Crie uma classe abstrata chamada CartaoWeb. Essa classe representa todos os tipos de cartões web e conterá apenas um atributo: destinatário (String). Nessa classe você deverá também declarar o método "public abstract void showMessage()". Crie classes filhas da classe CartaoWeb: DiaDosNamorados, Natal, Aniversario. Cada uma dessas classes, deve conter um método construtor que receba o nome do destinatário do cartão. Cada classe também deve implementar o método showMessage(), mostrando uma mensagem ao usuário com seu nome e que seja específica para a data de comemorativa do cartão. Por exemplo, essa poderia ser uma mensagem de um cartão de dia dos namorados:

"Querida Maria,

Feliz Dia dos Namorados!
Espero que esse tenha sido o único cartão do dia dos namorados que tenha ganhado nessa data! ;-) "

De todo meu coração,
João"

No método main de uma classe qualquer, crie um array de CartaoWeb. Insira de forma alternada, instâncias dos 3 tipos de cartões nesta array.
Após, use um laço for para exibir as mensagens deste cartão chamando o método showMessage(). Em que linha(s) acontece polimorfismo nesse código?

Estudo de caso usando polimorfismo

Uma empresa paga seus funcionários semanalmente. Os funcionários são de quatro tipos:

Funcionários assalariados recebem salários fixos semanais independentemente do número de horas trabalhadas

Funcionários que trabalham por hora são pagos da mesma forma e recebem horas extras (1.5 vezes sua taxa de salario por hora) por todas as horas além das 40 horas normais

Funcionários comissionados recebem porcentagem sobre suas vendas

Funcionários assalariados/comissionados recebem um salário base mais uma porcentagem sobre suas vendas

Obs: Para o período atual a empresa decidiu recompensar os assalariados/comissionados adicionando 10% ao salário-base. A empresa quer escrever um aplicativo Java que realiza os cálculos de folha de pagamento polimorficamente.

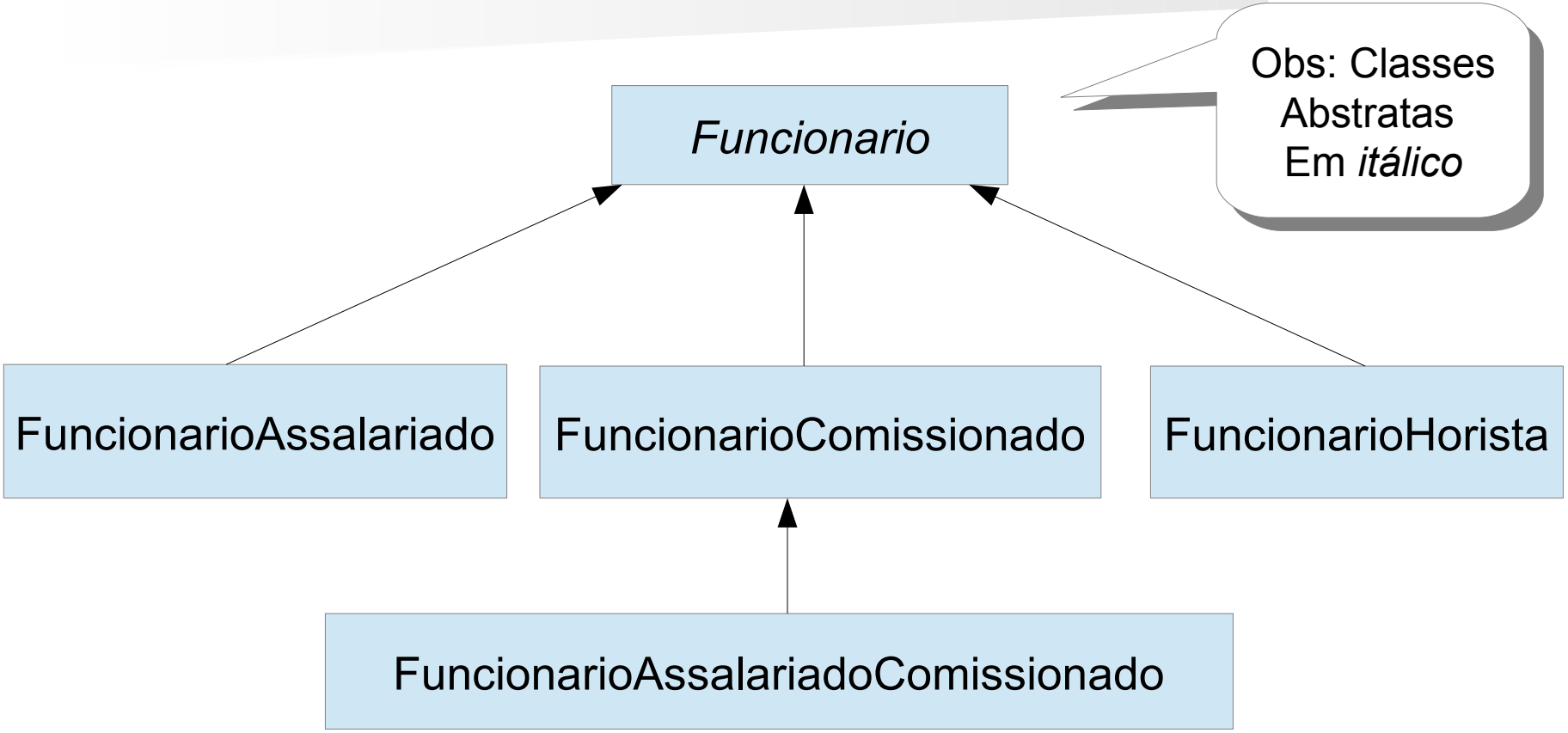
Estudo de caso usando polimorfismo

Utilizamos uma classe *abstrata* Funcionario para representar o conceito geral de um funcionario

E as classes que estendem Funcionario são:

- FuncionarioAssalariado
- FuncionarioComissionado
- FuncionarioHorista
- FuncionarioAssalariadoComissionado

Estudo de caso usando polimorfismo



Estudo de caso usando polimorfismo

Superclasse abstrata Funcionario:

- Fornece métodos ganhos() e toString()
- Além dos métodos get e set que manipulam as variáveis de instância
- Método ganhos() declarado como abstrato
  - Todas as subclasses devem implementar

Estudo de caso usando polimorfismo

```
//superclasse abstrata Funcionario
public abstract class Funcionario
{
    private String nome;
    private String sobrenome;
    private String id;

    //construtor com tres argumentos
    public Funcionario(String nome, String sobrenome, String id){
        this.nome=nome;
        this.sobrenome=sobrenome;
        this.id=id;
    }
```

```java
//configura o nome
public void setNome(String nome){
    this.nome=nome;
}
//retorna o nome
public String getNome(){
    return nome;
}
//configura o sobrenome
public void setSobrenome(String sobrenome){
    this.sobrenome = sobrenome;
}
//retorna o sobrenome
public String getSobrenome(){
    return sobrenome;
}
```

```java
    //configura a identificacao
    public void setId(String id){
        this.id = id;
    }
    //retorna a identificacao
    public String getId(){
        return id;
    }
    //retorna representacao string do objeto Funcionario
    public String toString(){
        return String.format("%s %s \nIdentidade: %s",
            getNome(),getSobrenome(), getId());
    }

    //metodo abstrato sobrescrito por subclasses concretas
    public abstract double ganhos();//nenhuma implementacao aqui

}
```

Classe FuncionarioAssalariado

```java
public class FuncionarioAssalariado extends Funcionario
{
    private double salarioSemanal;

    //construtor com quatro argumentos
    public FuncionarioAssalariado(String nome, String sobrenome, String id,
        double salario){
        super(nome, sobrenome, id);
        setSalarioSemanal(salario);
    }
    //configura e valida o salario
    public void setSalarioSemanal(double salario){
        salarioSemanal = salario < 0.0 ? 0.0 : salario;
    }
    //retorna o salario
    public double getSalarioSemanal(){
        return salarioSemanal;
    }
```

```java
    //sobrescreve o metodo abstrato
    public double ganhos(){
        return getSalarioSemanal();
    }
    //sobrescreve toString
    public String toString(){
        return String.format("Funcionario Assalariado: %s \n%s: R$%,.2f",
            super.toString(), "salario semanal: ", getSalarioSemanal());
    }
}
```

Classe FuncionarioHorista

```java
public class FuncionarioHorista extends Funcionario
{
    private double salarioHora; //salario por hora
    private double horas; //horas trabalhadas na semana
    //construtor com cinco argumentos
    public FuncionarioHorista(String nome, String sobrenome, String id,
        double salHora, double horas){
        super(nome, sobrenome, id);
        setSalarioHora(salHora);
        setHoras(horas);
    }
    //configura remuneracao
    public void setSalarioHora(double sal){
        salarioHora = (sal < 0.0) ? 0.0 : sal;
    }
    //retorna remuneracao
    public double getSalarioHora(){
        return salarioHora;
    }
```

```java
//configura horas trabalhadas
public void setHoras(double horas){
    this.horas = (horas >= 0.0 && horas <= 168.0) ? horas : 0.0;
}
//retorna horas trabalhadas
public double getHoras(){
    return horas;
}
//sobrescreve o metodo abstrato
public double ganhos(){
    if(getHoras() <= 40)
        return getSalarioHora()*getHoras();
    else
        return 40*getSalarioHora()+(getHoras()-40)*getSalarioHora()*1.5;
}
```

```java
    //sobrescreve a representacao String do objeto
    public String toString(){
        return String.format("Funcionario Horista: %s \n %s: R$%,.2f; %s %,.2f",
            super.toString(), "salario por hora", getSalarioHora(),
            "horas trabalhadas", getHoras());
    }
}
```

Classe FuncionarioComissionado

```java
public class FuncionarioComissionado extends Funcionario
{
    private double vendasBrutas; //vendas brutas semanais
    private double comissao; //porcentagem da comissao
    //construtor com cinco argumentos
    public FuncionarioComissionado(String nome, String sobrenome, String id,
        double vendas, double comissao){
        super(nome, sobrenome, id);
        setVendasBrutas(vendas);
        setComissao(comissao);
    }
    //valida e configura taxa de comissao
    public void setComissao(double comissao){
        this.comissao = (comissao > 0.0 && comissao < 1.0) ? comissao : 0.0;
    }
    //retorna taxa de comissao
    public double getComissao(){
        return comissao;
    }
```

```java
    //valida e configura vendas brutas
    public void setVendasBrutas(double vendas){
        vendasBrutas = (vendas < 0.0) ? 0.0 : vendas;
    }
    //retorna vendasBrutas
    public double getVendasBrutas(){
        return vendasBrutas;
    }
    //calcula rendimentos; sobrescreve o metodo abstrato
    public double ganhos(){
        return getComissao()*getVendasBrutas();
    }
    public String toString(){
        return String.format("%s: %s\n%s: R$%,.2f; %s: %,.2f",
        "Funcionario Comissionado", super.toString(),
        "vendas brutas", getVendasBrutas(),
        "porcentagem de comissao", getComissao());
    }
}
```

Classe FuncionarioAssalariadoComissionado

```java
//subclasse concreta indireta
public class FuncionarioAssalariadoComissionado extends FuncionarioComissionado
{
    private double salarioBase; //salario base por semana

    public FuncionarioAssalariadoComissionado(String nome, String sobrenome, String id,
        double vendas, double comissao, double salario){
        super(nome, sobrenome, id, vendas, comissao);
        setSalarioBase(salario);
    }
    //valida e configura salario base
    public void setSalarioBase(double salario){
        salarioBase = (salario < 0.0)? 0.0 : salario;
    }
    //retorna salario base
    public double getSalarioBase(){
        return salarioBase;
    }
```

```java
    //calcula os vencimentos; sobrescreve metodo de FuncionarioComissionado
    public double ganhos(){
        return salarioBase + super.ganhos();
    }
    //retorna representacao string do objeto
    public String toString(){
        return String.format("%s %s; %s: $%,.2f",
            "(Assalariado)", super.toString(),
            "salario base", getSalarioBase());
    }
}
```

Classe de Teste

Estudo de caso com polimorfismo

Primeiro vamos criar objetos e processá-los separadamente:

```java
public class TesteSistemaPagamento
{
    public static void main (String[] args){
        FuncionarioAssalariado fAssalariado =
            new FuncionarioAssalariado("Joao", "da Silva", "111-111-111", 800.00);
        FuncionarioHorista fHorista =
            new FuncionarioHorista("Maria", "de Souza", "222-222-222", 16.75, 40);
        FuncionarioComissionado fComissionado =
            new FuncionarioComissionado("Ana", "Pereira", "333-333-333", 10000, .06);
        FuncionarioAssalariadoComissionado fAssalariadoComissionado =
            new FuncionarioAssalariadoComissionado(
                "Paulo", "Andrade", "444-444-444", 5000, .04, 300);

        System.out.println("Funcionarios processados individualmente: \n");
```

```java
System.out.printf("%s\n%s: R$%,.2f\n\n",
    fAssalariado, "ganhou", fAssalariado.ganhos());
System.out.printf("%s\n%s: R$%,.2f\n\n",
    fHorista, "ganhou", fHorista.ganhos());
System.out.printf("%s\n%s: R$%,.2f\n\n",
    fComissionado, "ganhou", fComissionado.ganhos());
System.out.printf("%s\n%s: R$%,.2f\n\n",
    fAssalariadoComissionado, "ganhou", fAssalariadoComissionado.ganhos());
```

Estudo de caso com polimorfismo

Depois vamos utilizar um array e processar os objetos polimorficamente:

```java
Funcionario[] funcionarios = new Funcionario[4];

funcionarios[0] = fAssalariado;
funcionarios[1] = fHorista;
funcionarios[2] = fComissionado;
funcionarios[3] = fAssalariadoComissionado;

System.out.println("Funcionarios processados polimorficamente: \n");
```

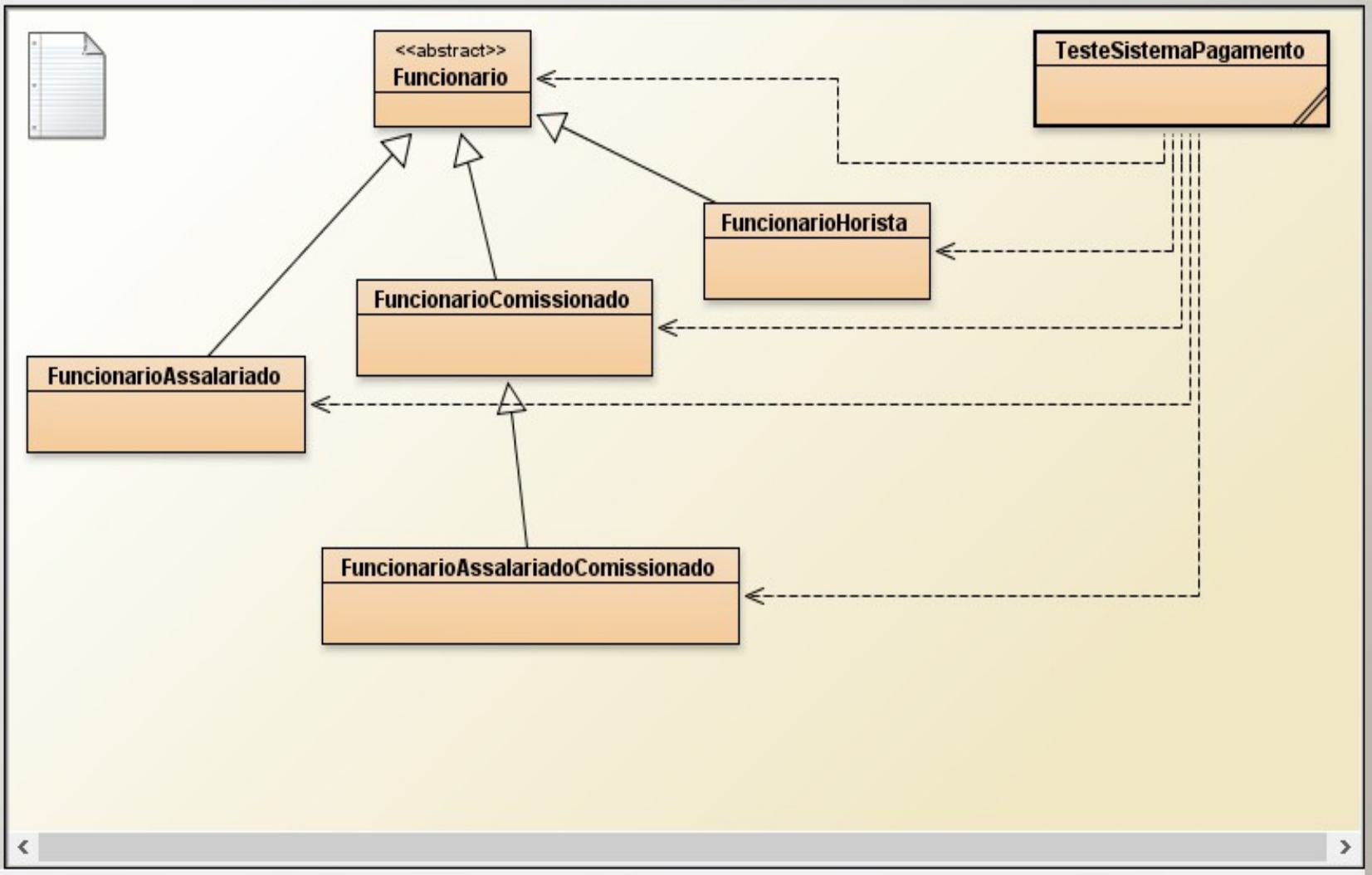
```java
//processa genericamente cada elemento de funcionarios
for (Funcionario funcAtual : funcionarios) {

    System.out.println(funcAtual);//invoca toString

    if (funcAtual instanceof FuncionarioAssalariadoComissionado){
        //downcast da referencia de Funcionario para
        //referencia a FuncionarioAssalariadoComissionado
        FuncionarioAssalariadoComissionado fAC =
            (FuncionarioAssalariadoComissionado) funcAtual;

        fAC.setSalarioBase(1.10*fAC.getSalarioBase());

        System.out.printf("novo salario base com 10%% de aumento e: R$%,.2f \n",
            fAC.getSalarioBase());
    }

    System.out.printf("ganhou R$%,.2f\n\n", funcAtual.ganhos());
}
```

BlueJ: AbstrataPolimorfismo

Project Edit Tools View Help

New Class...

Compile



Compiling... Done.

```
Funcionarios processados individualmente:

Funcionario Assalariado: Joao da Silva
Identidade: 111-111-111
salario semanal: : R$800,00
ganhou: R$800,00

Funcionario Horista: Maria de Souza
Identidade: 222-222-222
 salario por hora: R$16,75; horas trabalhadas 40,00
ganhou: R$670,00

Funcionario Comissionado: Ana Pereira
Identidade: 333-333-333
vendas brutas: R$10.000,00; porcentagem de comissao: 0,06
ganhou: R$600,00

(Assalariado) Funcionario Comissionado: Paulo Andrade
Identidade: 444-444-444
vendas brutas: R$5.000,00; porcentagem de comissao: 0,04; salario base: $300,0
ganhou: R$500,00
```

```
Funcionarios processados polimorficamente:

Funcionario Assalariado: Joao da Silva
Identidade: 111-111-111
salario semanal: : R$800,00
ganhou R$800,00

Funcionario Horista: Maria de Souza
Identidade: 222-222-222
 salario por hora: R$16,75; horas trabalhadas 40,00
ganhou R$670,00

Funcionario Comissionado: Ana Pereira
Identidade: 333-333-333
vendas brutas: R$10.000,00; porcentagem de comissao: 0,06
ganhou R$600,00

(Assalariado) Funcionario Comissionado: Paulo Andrade
Identidade: 444-444-444
vendas brutas: R$5.000,00; porcentagem de comissao: 0,04; salario base: $300,00
novo salario base com 10% de aumento e: R$330,00
ganhou R$530,00
```